MANUEL OFFICIEL DE RÉFÉRENCE • ÉDITION 2026

SmartWallet
Guide Complet d'Utilisation & de Gestion

Plateforme de Gestion Budgétaire Intelligente (Stack MERN, OCR Tesseract & Analyse IA)

Ce guide fournit toutes les instructions opérationnelles, fonctionnelles et techniques pour installer, utiliser, exploiter et administrer l'application **SmartWallet** en parfaite conformité avec les spécifications du projet.

ARCHITECTURE **MERN Full-Stack**	MOTEUR OCR **Tesseract.js (ENG+FRA)**	MODULE IA **Gemini & Moteur Expert**
GESTION BUDGÉTAIRE **Plafonds & Alertes (80%/100%)**	ÉPARGNE **Objectifs Indépendants**	DEVISES **MAD, USD, EUR, GBP, CAD**

Table des Matières

1. Présentation Générale de SmartWallet — Page 3
   - 1.1 Objectifs Clés et Périmètre Fonctionnel
   - 1.2 Ce que l'application fait (et ne fait pas)
2. Installation & Démarrage Rapide — Page 3
   - 2.1 Prérequis Techniques
   - 2.2 Configuration des Environnements (.env)
   - 2.3 Lancement du Serveur Backend et du Client Frontend
3. Guide d'Utilisation Pas-à-Pas — Page 4
   - 3.1 Authentification et Gestion du Profil
   - 3.2 Tableau de Bord Financier Interactif
   - 3.3 Gestion des Revenus, Dépenses et Export CSV
   - 3.4 Budgets Mensuels et Système d'Alertes Actives
   - 3.5 Objectifs d'Épargne sans Double Comptage
   - 3.6 Scanner de Reçus par OCR & Confirmation Utilisateur
   - 3.7 Rapports Mensuels IA en 5 Sections
   - 3.8 Personnalisation des Catégories de Dépenses
4. Le Scénario de Démonstration (Cas de Yassine) — Page 6
   - 4.1 Utilisation du Bouton Démo en 1 Clic
   - 4.2 Déroulé chronologique du cas d'évaluation
5. Architecture Technique & Administration — Page 7
   - 5.1 Modèles de Données Mongoose
   - 5.2 API REST Express & Endpoints
   - 5.3 Pipeline OCR & Moteur d'Analyse IA
6. Guide de Déploiement en Production — Page 8
7. Dépannage & FAQ — Page 8

Conseil Rapide pour l'Évaluation

Pour une démonstration immédiate, connectez-vous et cliquez sur le bouton violet **"Load PDF Demo"** dans l'en-tête : toutes les données réalistes (salaire, budgets, ticket Marjane, épargne, rapport IA) seront instantanément chargées !

1. Présentation Générale de SmartWallet

SmartWallet est une application web moderne conçue pour permettre aux particuliers de maîtriser leur budget mensuel, catégoriser leurs transactions financières, se prémunir contre les dépassements de dépenses et faire fructifier leur épargne personnelle sans complexité inutile.

1.1 Objectifs Clés

1. **Simplifier le suivi financier personnel**
   Offrir une interface fluide et réactive pour saisir, catégoriser, rechercher et visualiser l'ensemble des flux d'argent.
2. **Améliorer la conscience budgétaire**
   Combiner des alertes en temps réel à 80% et 100% avec des rapports analytiques mensuels générés par Intelligence Artificielle.
3. **Automatiser la saisie des reçus par OCR**
   Éviter la saisie manuelle grâce à la reconnaissance optique de caractères (Tesseract.js) couplée à la suggestion de catégorie.

1.2 Périmètre Fonctionnel

FONCTIONNALITÉS INCLUSES (IN-SCOPE)	HORS PÉRIMÈTRE (OUT-OF-SCOPE)
Suivi rigoureux des revenus et dépenses personnels	Connexions bancaires ouvertes (Open Banking)
Plafonds budgétaires globaux et catégoriels mensuels	Virements de fonds ou transferts d'argent
Objectifs d'épargne avec suivi des versements indépendants	Passerelle de paiement en ligne ou cartes virtuelles
OCR de tickets de caisse & proposition de catégorie	Conseil financier d'investissement spéculatif
Rapports financiers mensuels IA stricts & historique	Multi-devises simultanées au sein d'un même compte

> ℹ️ **Règle d'Or de l'Épargne**
> Verser de l'argent vers un objectif d'épargne (ex: 1 000 MAD vers l'objectif PC) **ne compte pas comme une dépense**. SmartWallet maintient une séparation stricte entre les dépenses et l'épargne pour éviter tout double-comptage.

2. Installation & Démarrage Rapide

SmartWallet est structuré en deux parties : API Express (`server/`) et client React Vite (`client/`).

- **Prérequis** : Node.js v18+, NPM v9+, MongoDB local sur `27017` (ou MongoDB Atlas).
- **Configuration Backend** : Fichier `server/.env` :

```
PORT=5001
MONGODB_URI=mongodb://127.0.0.1:27017/smartwallet
JWT_SECRET=smartwallet_super_secret_jwt_key_2026_finance_secure
GEMINI_API_KEY=
CLIENT_URL=http://localhost:5173
NODE_ENV=development
```

A. **Lancer le Serveur Backend**
   Exécutez dans un terminal : `cd server && npm run dev` (démarre sur **http://localhost:5001**).

B

Lancer le Client Frontend

Exécutez dans un second terminal : `cd client && npm run dev` (accessible sur **http://localhost:5173**).

3. Guide d'Utilisation Pas-à-Pas

3.1 Inscription et Choix de la Devise

Sur **http://localhost:5173**, accédez à l'inscription via *"Register here"* :

- Renseignez votre nom, email et mot de passe (6 caractères minimum).
- Sélectionnez votre devise de référence : **MAD**, **USD**, **EUR**, **GBP** ou **CAD**.
- Les catégories par défaut (Groceries, Transport, Dining, Salary...) sont automatiquement créées pour votre compte.

3.2 Le Tableau de Bord Financier (Dashboard)

- **Cartes Métriques** : Revenus totaux, Dépenses totales, Solde net disponible, et Pourcentage du budget utilisé.
- **Bannières d'Alertes** : Messages d'avertissement automatiques dès que le budget global ou une catégorie atteint 80% ou dépasse 100%.
- **Graphique Donut Recharts** : Répartition visuelle des dépenses par catégorie avec infobulles interactives.
- **Barre de Progression Globale** : Jauge en temps réel illustrant le montant restant et le statut (vert, orange, rouge).
- **Dernières Transactions** : Liste des mouvements récents avec badge d'accès direct au reçu scanné.
- **Sélecteur de Mois** : Permet de basculer sur n'importe quel mois pour consulter son historique.

3.3 Gestion des Revenus, Dépenses & Export CSV

- **Ajout Manuel** : Cliquez sur *"New Transaction"*, choisissez Dépense/Revenu, le montant, la catégorie, la date et la méthode de paiement.
- **Filtres & Recherche** : Filtrez par plage de dates, type ou catégorie, ou tapez un mot-clé dans la barre de recherche.
- **Exportation CSV** : Cliquez sur *"Export CSV"* pour télécharger l'historique complet compatible Excel.
- **Visualisation de Reçu** : Cliquez sur le bouton vert *"View"* pour ouvrir la photo originale du ticket.

3.4 Budgets Mensuels & Alertes Actives

Sur l'écran **Monthly Budgets** :

- **Plafond Global** : Limite globale mensuelle (ex: 4 000 MAD).
- **Plafonds par Catégorie** : Limites spécifiques (ex: 1 500 MAD Épicerie, 600 MAD Transport).
- **Calculs Réels** : Le système agrège les dépenses de transactions pour calculer la consommation en temps réel.

CONSOMMATION	STATUT	COMPORTEMENT DU SYSTÈME
Moins de 80%	MAÎTRISÉ	Barre verte, solde disponible suffisant
Entre 80% et 99%	ATTENTION (WARNING)	Bannière jaune d'avertissement proactive
100% et plus	DÉPASSEMENT (EXCEEDED)	Bannière rouge critique indiquant le montant excédentaire

3.5 Objectifs d'Épargne Personnels (Savings Goals)

Sur l'écran **Savings Goals** :

- **Créer un Objectif** : Définissez un nom (ex: "Nouveau Laptop"), un montant cible (ex: 8 000 MAD) et une date cible.
- **Enregistrer un Versement** : Cliquez sur *"Record Contribution"* pour allouer une somme (ex: 500 MAD).
- **Garantie de Non-Double Comptage** : L'épargne est dissociée des dépenses courantes pour préserver l'exactitude des bilans.

3.6 Scanner de Reçus par OCR & Validation Utilisateur

1. **Téléversement du ticket**
   Glissez-déposez l'image de votre reçu sur la zone dédiée (PNG, JPG, WEBP).
2. **Extraction Tesseract.js & Catégorisation IA**
   Le serveur extrait le Commerçant (Marjane), la Date, le Montant Total (285.00 MAD) et suggère la catégorie (Groceries).
3. **Modale d'Inspection & Validation**
   Une modale interactive s'affiche : vérifiez les champs, modifiez si besoin, puis cliquez sur **"Confirm & Save Expense"**.

3.7 Rapports Mensuels par Intelligence Artificielle

L'écran **AI Monthly Reports** génère une analyse financière experte en **5 sections structurées** :

> **Règle de Fiabilité Absolue**
> Tous les montants, totaux et pourcentages sont **calculés mathématiquement par Express.js** avant d'être transmis à l'IA. L'IA ne peut jamais inventer de chiffres financiers.

1. **Aperçu Mensuel (Monthly Overview)** : Tableau chiffré des revenus, dépenses, taux d'épargne réel et état d'utilisation du budget global.
2. **Tendances de Dépenses (Spending Patterns)** : Identification des catégories majeures et comparaison en pourcentage avec le mois précédent (ou notification explicite d'historique insuffisant).
3. **Points d'Attention Budgétaires (Budget Concerns)** : Mise en exergue des catégories qui approchent ou dépassent leurs plafonds alloués.
4. **Observations Positives (Positive Observations)** : Valorisation des catégories où les dépenses sont restées maîtrisées et de la capacité d'épargne maintenue.
5. **Recommandations Pratiques Chiffrées (Recommendations)** : Conseils d'optimisation concrets basés sur vos montants réels.

Persistance : Chaque rapport est sauvegardé dans MongoDB et peut être réouvert immédiatement sans ré-exécuter l'IA.

3.8 Gestion des Catégories de Dépenses

- Consultez les catégories natives (Groceries, Transportation, Housing...).
- Créez vos propres catégories personnalisées avec nom, type et couleur thématique.

4. Le Scénario de Démonstration (Cas de Yassine)

La page 12 du sujet décrit l'histoire de **Yassine**. Cliquez sur **"Load PDF Demo"** dans l'en-tête pour charger instantanément ce scénario de référence :

ÉTAPE	ACTION RÉALISÉE DANS LE SCÉNARIO	VÉRIFICATION DANS L'APPLICATION
1. Profil	Configuration du compte avec devise **MAD**	Badge MAD visible dans toute l'interface
2. Revenus & Budget	Salaire (9 000 MAD) et budget mensuel de **4 000 MAD** (Épicerie: 1 500 MAD, Transport: 600 MAD)	Page *Budgets* : jauges et plafonds correctement configurés
3. Épargne PC	Objectif de **8 000 MAD** pour le PC avec versement initial de 1 000 MAD	Page *Savings* : 1 500 MAD économisés (18.8%)
4. Reçu Scanné	Achat de courses au supermarché Marjane : **285,00 MAD** extrait par OCR Tesseract	Page *Receipts* : image Marjane et transaction Groceries liée
5. Dashboard	Le transport atteint 510 MAD / 600 MAD (85%), déclenchant l'alerte proactive	Page *Dashboard* : bannière jaune d'avertissement Transport
6. Rapport IA	Génération de l'analyse mensuelle comparée au mois précédent	Page *Reports* : rapport complet en 5 sections avec historique antérieur

> 💡 **Image de Reçu Échantillon Incluse**
> Un ticket de caisse réel a été généré et stocké dans `server/uploads/receipts/sample-receipt.png`. Vous pouvez tester le scanner OCR en direct en téléversant cette image !

5. Architecture Technique & Administration

5.1 Modèles de Données Mongoose

- **User** : `name`, `email` (unique), `password` (haché bcrypt), `preferredCurrency` (MAD).
- **Transaction** : `user`, `type`, `amount`, `category`, `date`, `description`, `receipt`, `paymentMethod`, `notes`.
- **Budget** : `user`, `month`, `overallLimit`, `categoryLimits: [{ category, limit }]`. Index `{ user: 1, month: 1 }`.
- **SavingsGoal** : `user`, `name`, `targetAmount`, `currentAmount`, `targetDate`, `contributions`.
- **Receipt** : `user`, `imageUrl`, `originalFilename`, `ocrRawText`, `extractedData`, `status`.
- **MonthlyReport** : `user`, `month`, `statistics`, `aiAnalysis`, `aiModelUsed`.

5.2 Routes Clés de l'API REST Express

MÉTHODE	ENDPOINT	RÔLE & DESCRIPTION
POST	/api/auth/register & /login	Authentification JWT & création de compte
GET / POST	/api/transactions	Recherche, filtres multi-critères, ajout dépense/revenu
GET	/api/transactions/export/csv	Exportation du fichier CSV des transactions
GET / POST	/api/budgets	Consultation et paramétrage du budget et des limites
GET / POST	/api/savings	Gestion des objectifs d'épargne
POST	/api/receipts/upload & /:id/confirm	Téléversement OCR et confirmation de dépense liée
POST / GET	/api/reports/generate & /:month	Génération et réouverture du rapport mensuel IA
POST	/api/demo/seed	Initialisation du scénario d'évaluation Yassine

6. Guide de Déploiement en Production

SmartWallet est prêt pour un déploiement cloud gratuit selon les recommandations du sujet :

- **Base de Données (MongoDB Atlas)** : Créez un cluster gratuit (M0 Free Tier), configurez l'utilisateur et l'accès réseau (0.0.0.0/0), puis insérez l'URI de connexion dans `MONGODB_URI`.
- **Backend (Render ou Railway)** : Liez votre dépôt GitHub, définissez le répertoire racine `server`, commande de build `npm install`, commande de démarrage `node src/server.js`. Variables : `PORT=5001`, `MONGODB_URI`, `JWT_SECRET`, `CLIENT_URL`.
- **Frontend (Vercel ou Netlify)** : Liez le sous-dossier `client`, commande de build `npm run build`, répertoire de sortie `dist`.

7. Dépannage & FAQ

> **❓ Erreur Port 5000 / 5001 Occupé (EADDRINUSE)**
> SmartWallet utilise le port **5001** pour éviter les conflits avec AirPlay ou d'autres applications. Vous pouvez modifier la variable `PORT` dans `server/.env` à tout moment.

> **❓ Que faire si l'image d'un reçu est floue ?**
> Tesseract.js extrait le maximum de texte possible. Si certains champs ne sont pas lisibles, SmartWallet ouvre automatiquement la **modale de validation** afin que vous puissiez ajuster manuellement le montant ou le commerçant avant l'enregistrement.

> **❓ L'IA peut-elle se tromper dans les calculs budgétaires ?**
> **Non.** Tous les totaux, pourcentages et dépassements sont calculés par MongoDB et Express.js via des formules mathématiques déterministes. L'IA se contente de rédiger l'interprétation qualitative à partir de ces chiffres validés.